Discrete-Event Simulation Project

M/M/c/k Queueing System

Theory and Simulation — Technical Report

A discrete-event simulation of a finite-capacity, multi-server Markovian queue, with derivation of performance metrics via Little's Law.

Author: Ahmed Hossam
Mustafa Tag Eldin Hossam Elden Ahmed

Implementation: Python 3 (standard library)

Date: June 2026

Contents

1 Introduction 1
1.1 Objectives 1
2 Theoretical Background 1
2.1 Kendall's Notation 1
2.2 The Exponential Distribution 1
2.3 The M/M/c/k Model 2
2.4 Traffic Intensity 2
2.5 Steady-State Distribution (Analytical Reference) 2
2.6 Little's Law 2
3 Simulation Design 3
3.1 Discrete-Event Approach 3
3.2 State Representation 3
3.3 Main Event Loop 3
4 Implementation 4
4.1 Module Overview 4
4.2 Input Validation 4
4.3 Generating Random Times 5
4.4 The Arrival Branch 5
4.5 The Departure Branch 5
4.6 Computing the Metrics 5
5 Example Results 6
5.1 High-Traffic Scenario (with rejections) 6
5.2 Normal-Traffic Scenario (no rejections) 6
5.3 Interpretation 7
6 Conclusion 7
6.1 Possible Extensions 7

1 Introduction

Queueing systems are everywhere: customers waiting at a bank, packets buffered in a router, jobs queued for a CPU. A **queueing model** is a mathematical abstraction of such a system, describing how units (customers) arrive, wait, are served, and leave. This report studies the **M/M/c/k** queue — a multi-server system with finite capacity — and presents a discrete-event simulation that reproduces its dynamics and estimates its key performance measures.

The accompanying program, `mmck_simulation.py`, is written in pure Python using only the standard library. It prompts the user for the model parameters, runs a time-driven event simulation, prints a chronological event log, and finally reports four performance metrics derived from the recorded timings.

1.1 Objectives

- Explain the theory behind the M/M/c/k queue and its place in Kendall's notation.
- Describe the discrete-event simulation strategy used in the code.
- Map each part of the theory onto the corresponding implementation.
- Present and interpret example simulation results.

2 Theoretical Background

2.1 Kendall's Notation

Queueing systems are classified using **Kendall's notation**, written as $A \,/\, B \,/\, c \,/\, k$, where each field describes one aspect of the system:

Field	Value	Meaning
A	`M`	Arrival process is **Markovian** — interarrival times are exponentially distributed (Poisson arrivals).
B	`M`	Service process is **Markovian** — service times are exponentially distributed.
c	`c`	Number of **parallel, identical servers**.
k	`k`	**System capacity** — the maximum number of customers allowed in the system (in service + waiting) at once.

The "M" stands for **memoryless** (Markovian): the exponential distribution is the only continuous distribution whose future is independent of the past, which is what makes these systems analytically tractable.

2.2 The Exponential Distribution

Both arrivals and service follow the exponential distribution. If events occur at rate λ, the time between consecutive events T has probability density

$$f(t) = \lambda e^{-\lambda t}, \quad t \geq 0,$$

with mean $E[T] = 1/\lambda$. In the model:

- Customers arrive at rate λ, so the **mean interarrival time** is $1/\lambda$.
- Each server completes work at rate μ, so the **mean service time** is $1/\mu$.

2.3 The M/M/c/k Model

The system has c servers and room for at most k customers in total. Behaviour depends on the number of customers n currently present:

Condition	Action
$n < c$	An arriving customer finds a free server and enters service immediately.
$c \le n < k$	All servers are busy; the arriving customer joins the waiting queue.
$n = k$	The system is full; the arriving customer is **rejected** (blocked) and lost.

Because capacity is finite, the system is always **stable** — even when $\lambda > c\mu$ it never grows without bound; excess load simply shows up as a higher rejection rate.

2.4 Traffic Intensity

A useful summary parameter is the **offered load** and the per-server **utilisation**:

$$a = \frac{\lambda}{\mu}, \qquad \rho = \frac{\lambda}{c\mu}.$$

Here a is the offered load in Erlangs, and ρ is the fraction of server capacity demanded. When $\rho < 1$ the servers can, on average, keep up with arrivals; when $\rho \ge 1$ the queue and rejections grow.

2.5 Steady-State Distribution (Analytical Reference)

For an M/M/c/k queue the long-run probability P_n of finding exactly n customers in the system is, with $a = \lambda/\mu$,

$$P_n = \begin{cases} \dfrac{a^n}{n!} P_0 & \text{for } \; 0 \le n \le c \\ \dfrac{a^n}{c!\, c^{n-c}} P_0 & \text{for } \; c < n \le k \end{cases}$$

where P_0 normalises the distribution so that $\sum_{n=0}^{k} P_n = 1$. The **blocking probability** — the chance an arrival is rejected — is exactly P_k. These closed-form results provide a reference against which the simulation's empirical estimates can be sanity-checked.

2.6 Little's Law

The cornerstone result connecting the metrics is **Little's Law**. For any stable system in steady state, the average number of customers L equals the effective arrival rate λ_{eff} times the average time spent W:

$$L = \lambda_{\text{eff}} \cdot W.$$

Because rejected customers never enter the system, the **effective** arrival rate $\lambda_{\text{eff}} = \lambda(1 - P_k)$ is what drives the metrics. The simulation applies Little's Law in both forms:

$$N_s = \lambda_{\text{eff}} \cdot T_s, \qquad N_q = \lambda_{\text{eff}} \cdot T_q,$$

linking the **time** metrics (T_s, T_q) to the **count** metrics (N_s, N_q).

3 Simulation Design

3.1 Discrete-Event Approach

Rather than advancing a clock in fixed steps, the program uses **discrete-event simulation** (DES): the clock jumps directly to the time of the next event. Only two event types exist:

1. **Arrival** — a new customer enters the system.
2. **Departure** — a busy server finishes serving its customer.

At each iteration the simulator compares the time of the next scheduled arrival against the earliest scheduled departure and advances the clock to whichever comes first. This is efficient and exact for Markovian systems, since nothing of interest happens between events.

> **Why DES?**
> Time-stepped simulation wastes effort on idle intervals and can miss or mis-order events that fall within the same step. Event-driven simulation processes exactly one state change at a time, in correct chronological order.

3.2 State Representation

The simulator keeps the following state:

Variable	Role
`clock`	Current simulation time.
`servers`	List of length c; each slot is either `None` (idle) or a tuple `(customer_id, service_end_time)`.
`queue`	FIFO list of customer IDs waiting for a free server.
`next_arrival`	Scheduled time of the next arrival event.
`arrival_times` / `service_start_times` / `departure_times`	Dictionaries recording, per customer, the timestamps needed for the metrics.
`rejected`	List of customer IDs blocked because the system was full.

3.3 Main Event Loop

The control flow of `run_simulation()` is:

Step	Description
1	Scan all busy servers to find the earliest `service_end_time` → `next_departure`.
2	If `next_arrival < next_departure`, process an **arrival**; otherwise process a **departure**.
3 (arrival)	Advance clock; if system full → reject, else assign a free server or enqueue. Schedule the following arrival.
3 (departure)	Advance clock; free the finishing server; if the queue is non-empty, pull the head customer into service.
4	Repeat until `clock >= sim_time`.

4 Implementation

4.1 Module Overview

The program is organised into four functions with a single responsibility each:

Function	Purpose
`get_inputs()`	Prompts for k, c, λ, μ, and simulation time, with validation and re-prompting on bad input.
`run_simulation()`	Runs the discrete-event loop and returns the event log plus per-customer timing records.
`calc_metrics()`	Computes T_s, T_q, N_s, N_q from the recorded timings.
`main()`	Orchestrates the flow: input → simulation → event log → metrics.

4.2 Input Validation

Each parameter is read inside a `while True` loop that re-prompts until a valid value is supplied, guarding against both type errors and out-of-range values:

```
while True:
    try:
        k = int(input("enter system capacity (k): "))
        if k < 1:
            print("  error: k must be >= 1")
            continue
        break
    except ValueError:
        print("  error: k must be an integer")
```

The same pattern enforces $c \geq 1$ (integer), and $\lambda, \mu, \text{sim\_time} > 0$ (float). This makes the program robust against malformed interactive input.

4.3 Generating Random Times

Exponential interarrival and service times are drawn with Python's `random.expovariate`, which takes the rate directly:

```
next_arrival = clock + random.expovariate(lam)   # next arrival
svc_time     = random.expovariate(mu)            # service duration
```

This is the direct computational realisation of the "M" (Markovian) assumptions described in Section 2.

4.4 The Arrival Branch

On an arrival the simulator checks total occupancy against capacity, then either rejects, assigns a free server, or enqueues:

```
total_in_system = sum(1 for s in servers if s is not None) + len(queue)

if total_in_system >= k:
    rejected.append(cid)                        # system full -> blocked
else:
    free = next((i for i, s in enumerate(servers) if s is None), None)
    if free is not None:
        svc_time = random.expovariate(mu)
        servers[free] = (cid, clock + svc_time)  # start service now
        service_start_times[cid] = clock
    else:
        queue.append(cid)                        # all busy -> wait
```

4.5 The Departure Branch

When a service completes, the server is freed and the head of the queue (if any) is immediately taken into service — the FIFO discipline:

```
servers[i] = None
if queue:
    next_cid = queue.pop(0)                      # FIFO
    svc_time = random.expovariate(mu)
    servers[i] = (next_cid, clock + svc_time)
    service_start_times[next_cid] = clock
```

4.6 Computing the Metrics

After the loop, `calc_metrics()` turns the recorded timestamps into the four performance measures. For each completed customer it computes the time in system and time in queue, averages them, estimates the effective arrival rate, and applies Little's Law:

```
ts_list = [departure_times[c] - arrival_times[c] for c in completed]
tq_list = [service_start_times[c] - arrival_times[c]
           for c in completed if c in service_start_times]

ts = sum(ts_list) / len(ts_list)                 # avg time in system
```

```
tq = sum(tq_list) / len(tq_list)                # avg time in queue

total_time = max(departure_times.values()) - min(arrival_times.values())
lam_eff = len(completed) / total_time           # effective arrival rate

ns = lam_eff * ts                               # Little's Law
nq = lam_eff * tq                               # Little's Law
```

Metric	Symbol	Definition
`ts`	T_s	Average time a served customer spends in the system (arrival $\longrightarrow$ departure).
`tq`	T_q	Average time a customer waits before service begins (arrival $\longrightarrow$ service start).
`ns`	N_s	Average number of customers in the system, $\lambda_{\text{eff}} \cdot T_s$.
`nq`	N_q	Average number of customers waiting in queue, $\lambda_{\text{eff}} \cdot T_q$.

5 Example Results

The following runs were produced by the program. Because arrivals and service times are random, exact numbers vary between runs, but the qualitative behaviour is consistent and matches queueing-theory expectations.

5.1 High-Traffic Scenario (with rejections)

Parameters $k = 3$, $c = 1$, $\lambda = 5$, $\mu = 2$, time $= 10$. Here the offered load $\rho = \lambda/(c\mu) = 5/2 = 2.5 \gg 1$, so the single server is heavily overloaded and most arrivals are blocked.

Quantity	Value
Customers arrived	58
Rejected (blocked)	41
Served	14
T_s — avg time in system	1.6416
T_q — avg time in queue	0.9481
N_s — avg number in system	2.3671
N_q — avg number in queue	1.3671

The blocking ratio of $41/58 \approx 71\%$ directly reflects the severe overload: with only one server and capacity 3, the system simply cannot absorb arrivals at $2.5\times$ its service rate.

5.2 Normal-Traffic Scenario (no rejections)

Parameters $k = 5$, $c = 2$, $\lambda = 2$, $\mu = 3$, time $= 10$. Now $\rho = 2/(2 \cdot 3) = 1/3 \ll 1$, so the two servers comfortably keep up and no customer is blocked.

Quantity	Value
Customers arrived	19
Rejected (blocked)	0
Served	18
T_s — avg time in system	0.4366
T_q — avg time in queue	0.0048
N_s — avg number in system	0.8690
N_q — avg number in queue	0.0095

Here $T_q \approx 0$: customers almost never wait, because a free server is nearly always available. Note that $T_s \approx 0.44 \approx 1/\mu = 0.33$ plus a small waiting component — i.e. the time in system is dominated by service time, exactly as expected for a lightly loaded system.

5.3 Interpretation

Comparing the two scenarios illustrates the central trade-off in queueing design:

- **Overload** ($\rho > 1$) produces high blocking and long waits — capacity must be increased (more servers c or faster service μ) or load reduced.
- **Light load** ($\rho < 1$) gives near-zero waiting at the cost of idle servers.

The finite capacity k converts what would be unbounded queue growth into a bounded system with a measurable loss (blocking) rate — the defining feature of the M/M/c/k model.

6 Conclusion

This project implements and analyses an M/M/c/k queueing system through discrete-event simulation. The theoretical model — Markovian arrivals and service, c parallel servers, finite capacity k — is realised faithfully in roughly 180 lines of clean, dependency-free Python.

The simulation advances an event clock between arrivals and departures, maintains explicit server and queue state, records per-customer timestamps, and derives the four standard performance metrics (T_s, T_q, N_s, N_q) using Little's Law. The example results behave exactly as theory predicts: heavy load yields high blocking and waiting, while light load yields negligible waiting.

6.1 Possible Extensions

- Average results over many independent replications to obtain confidence intervals.
- Compare empirical metrics against the analytical steady-state P_n and blocking probability P_k for validation.
- Support non-exponential distributions (G/G/c/k) or priority queue disciplines.
- Add warm-up removal to reduce initialisation bias in the estimates.